

Code Review Guidelines

[wiki/engineering/code-review-guidelines](#) · Updated 2026-08-01

Owner: Platform Engineering · POC: Nadia Hassan, Engineering Manager

Purpose

These guidelines describe how code review works at Northwind. They are maintained by the Platform team and apply to every repository. Good review keeps our code correct, maintainable, and shared across the team.

Review is not a gate to pass; it is a conversation that improves the change and spreads knowledge. Both authors and reviewers share responsibility for making it effective.

Why we review

Review catches defects early, spreads knowledge across the team so no one person is a single point of failure, and keeps the codebase consistent. It is one of the highest-leverage habits an engineering team can have.

Requirements

- Every pull request needs at least one approval before merge.
- CI must be green before merge; do not bypass required checks.
- Keep pull requests under 400 lines where possible.
- Reviewers should respond within one business day.

Why small pull requests

Small changes are reviewed faster, more thoroughly, and with fewer mistakes. If a change is large, split it into a series of reviewable steps rather than asking for one enormous review. A reviewer's attention does not scale with the size of the diff.

What reviewers look for

Focus review on correctness, tests, and readability.

- Correctness: does the change do what it claims, including edge cases?
- Tests: are there tests that would fail without the change?
- Readability: will someone understand this in six months?
- Security: are inputs validated and secrets kept out of the code?

What not to block on

Avoid blocking a change on personal style preferences that the formatter does not enforce. Prefer suggestions over demands for subjective matters, and let automated tooling handle formatting. Reserve blocking comments for genuine problems.

For authors

- Write a clear description explaining what changed and why.
- Keep each pull request focused on a single concern.
- Respond to feedback promptly and assume good intent.
- Do not merge your own change without the required approval.

- Make the diff easy to review: separate refactors from behaviour changes.

Writing a good description

A good description tells the reviewer what the change does, why it is needed, and anything they should pay special attention to. It saves the reviewer from reverse-engineering your intent from the diff.

For reviewers

- Be timely; a stalled review blocks your teammate.
- Be specific and kind; explain the reasoning behind a request.
- Distinguish between must-fix issues and optional suggestions.
- Approve once the must-fix items are addressed.
- Praise good work, not just problems.

Giving good feedback

Explain the why behind a comment so the author learns, not just complies. Mark optional suggestions clearly so the author knows what actually blocks the merge. Assume the author made reasonable choices and ask before asserting.

Worked example

An author opens a focused 180-line pull request with a clear description and tests. A reviewer responds the same day, flags one correctness issue as must-fix and two readability points as optional suggestions, and explains the reasoning for each. The author fixes the must-fix issue, adopts one

suggestion, replies to the other, and the reviewer approves. CI is green, so the author merges.

Frequently asked questions

Can I merge my own change? Only with the required approval from someone else.

What if a required check is wrong? Fix the check; do not bypass it.

How big is too big? Aim for under 400 lines. Split larger changes into reviewable steps.

Common mistakes

- Opening a huge pull request that mixes a refactor with a behaviour change.
- Blocking a merge over subjective style the formatter does not enforce.
- Letting a review sit for days and blocking a teammate.
- Bypassing a required check under deadline pressure.

Merging

Once a change has the required approval and a green pipeline, the author merges it. Required checks exist to protect everyone and must never be bypassed, even under deadline pressure.

Related documents

See the CI/CD Pipeline Standards for the checks that run on every pull request and the Deployment Guide for what happens after a change merges.

Review checklist

Use this quick checklist as a reviewer. It is not exhaustive, but it covers the issues that most often matter and are easiest to miss when skimming a diff.

- Does the change do what the description says, including the edge cases?
- Are there tests that would fail without the change, and do they pass with it?
- Will this be understandable to someone reading it in six months?
- Are inputs validated and are secrets kept out of the code and logs?
- Is the change appropriately small and focused on a single concern?
- Does it update any documentation or runbooks that the change makes stale?

If every box is ticked and CI is green, approve. If something is unclear, ask rather than assume.

Handling disagreements

Most review comments are resolved quickly, but occasionally an author and reviewer disagree. Keep the discussion focused on the change and the

reasoning, not on who is right.

- Explain the trade-off you see and ask for the other person's reasoning.
- If you cannot converge, bring in a third engineer or the code owner for a quick decision.
- Prefer the simpler option that is easier to change later when the evidence is balanced.
- Record the decision in the pull request so the reasoning is not lost.

Disagreements handled well are a strength: they surface assumptions and usually produce a better change than either person would have made alone.